

EFARMER – ALL IN ONE RESOURCE FOR A THRIVING FARM

A PROJECT

Submitted by

Deepak Yadav (2021000444)

Gaurav Kumar Singh (2021000823)

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering

Under the supervision of

Dr. Avinash Kumar Sharma

Assoc. Prof

Department of Computer Science & Engineering



**SHARDA
UNIVERSITY**
Beyond Boundaries



SHARDA SCHOOL OF ENGINEERING AND TECHNOLOGY

SHARDA UNIVERSITY, GREATER NOIDA – 201310

May, 2025

TABLE OF CONTENTS

DECLARATION.....	i
CERTIFICATE.....	ii
ACKNOWLEDGMENT.....	iii
LIST OF TABLES	iv
LIST OF FIGURES.....	v
SYMBOLS AND ABBREVIATIONS.....	vi
CHAPTER 1: INTRODUCTION.....	1
1.1 Overview of Indian agriculture challenges.....	2
1.2 Need for digital solutions like Efarmer.....	2
1.3 Objectives and Motivation.....	3
CHAPTER 2: LITERATURE SURVEY.....	4
2.1 Research on existing agriculture platforms.....	5
2.2 Studies on ML for disease detection.....	6
2.3 Chatbot use in farming support.....	7
CHAPTER 3: SYSTEM ANALYSIS.....	9
3.1 Requirements gathering.....	10
3.2 Functional vs non-functional requirements.....	10
3.3 System architecture and modules.....	11
CHAPTER 4: TECHNOLOGY STACK.....	13
4.1 HTML, CSS, JS (Front-End).....	14
4.2 Flask, Python, PyTorch (Back-End).....	14
4.3 Gemini API, OpenWeatherMap, Agmarket API.....	15
4.4 JSON for local data persistence.....	15
4.5 TailwindCSS for UI.....	16
CHAPTER 5: MODULE DESCRIPTION.....	17
5.1 Crop Guide.....	18
5.2 Disease Detection.....	18
5.3 Chatbot.....	19
5.4 Mandi Prices.....	19
5.5 Weather.....	19
5.6 Formus.....	20
5.7 News.....	20
CHAPTER 6: MODEL TRAINING FOR DISEASE DETECTION.....	21
6.1 Dataset used.....	22
6.2 Preprocessing.....	22
6.3 CNN architecture.....	23
6.4 Evaluation metrics.....	23
CHAPTER 7: IMPLEMENTATION.....	25
7.1 Directory Structure.....	26
7.2 Routing (Flask)	26
7.3 Upload & prediction logic.....	27
7.4 Frontend UX design (TailwindCSS, basic HTML).....	28
CHAPTER 8: TESTING.....	30
8.1 Unit Testing.....	31
8.2 Integration testing.....	31
8.3 Manual validation of disease predictions.....	32
CHAPTER 9: RESULTS.....	34
9.1 Screenshots.....	35

9.2 Model accuracy results.....	38
CHAPTER 10: CONCLUSION & FUTURE WOR.....	39
10.1 Summary of achivements.....	40
10.2 Future improvements.....	40
LIST OF PUBLICATION.....	42
REFERENCES.....	44

DECLARATION

I hereby declare that the project work entitled “Efarmer – All in one Resource for a Thriving Farm” submitted to Sharda University, Greater Noida is a record of an original work done by me under the guidance of Dr. Avinash Kumar Sharma and this research is being submitted to fulfil the requirements for the award of degree of Bachelor of Technology in Computer Science & Engineering.

The results embodied in this research work have not been submitted to any other university or institution for the award of any degree or diploma.

Place: Greater Noida

Signature of the Student-1

Date:

Signature of the Student-2

CERTIFICATE

This is to certify that the report entitled “Efarmer – All in one Resource for a Thriving Farm” submitted by Deepak Yadav (2021000444) and Gaurav Kumar Singh (2021000823) to Sharda University, towards the fulfilment of requirements of the degree of **Bachelor of Technology** is record of Bonafide final year Project work carried out by them in the Department of Computer Science and Engineering, School of Engineering and Technology, Sharda University.

The results/findings contained in this Project have not been submitted in part or full to any other University/Institute for award of any other Degree/Diploma.

Signature of Supervisor

Name: Dr. Avinash Kumar Sharma

Designation: Assoc. Professor (CSE)

Signature of Head of Department

Name: Dr. Sudeep Varshney

Place: Sharda University

Date:

Signature of External Examiner

Date:

ACKNOWLEDGEMENT

A major project is a golden opportunity for learning and self-development. We consider our self very lucky and honored to have so many wonderful people lead us through in completion of this project.

First and foremost, we would like to thank Dr. Sudeep Varshney, HOD, CSE who gave us an opportunity to undertake this project.

Our grateful thanks to Dr. Avinash Kumar Sharma for his guidance in my project work. Dr. Avinash Kumar Sharma, who despite being extraordinarily busy with academics, took time out to hear, guide and keep us on the correct path.

CSE department monitored our progress and arranged all facilities to make life easier. We choose this moment to acknowledge their contribution gratefully.

Signature of Students

Deepak Yadav (2021000444)

Gaurav Kumar Singh (2021000823)

LIST OF TABLES

Table 1: Literature Survey	7
Table 2: Functional Requirement.....	10
Table 3: Non-Functional Requirements.....	11
Table 4: Modules	11
Table 5: Tools Used	16
Table 6: Preprocessing	22
Table 7: Confusion Matrix	24
Table 8:Routing	26
Table 9:Testing Components	31
Table 10:Integration Scenarios Tested.....	32
Table 11:Results.....	32
Table 12:Test Accuracy by Class	35

LIST OF FIGURES

Figure 1:High-Level Architecture.....	11
Figure 2: CNN Architecture.....	23
Figure 3:Directory Structure	26
Figure 4:Image Upload Logic	27
Figure 5:Model Prediction Logic.....	28
Figure 6:Allowed Files	31
Figure 7:Upload form to submit a leaf image.....	35
Figure 8:Result page showing uploaded image and predicted disease (e.g., “Early Blight”)	36
Figure 9:Chatbot Responses	36
Figure 10:Mandi Prices Page	37
Figure 11:Weather Forecast.....	37
Figure 12:News Module	38
Figure 13:Crop Guide Navigation	38

SYMBOLS AND ABBREVIATIONS

LLMs	Large Language Models
BERT	Bidirectional Encoder Representations from Transformers
RoBERTa	Robustly Optimized BERT Approach
DLNN	Deep Learning Neural Network
ROC	Receiver Operating Characteristic Curve
GPT	Generative Pre-Training
AI	Artificial Intelligence
BARD AI	Bard Artificial Intelligence
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HPC	High-Performance Computing
SRS	System Requirement Specification
API	Application Programming Interface
RNN	Recurrent Neural Network
CNN	Convolutional Neural Network
NLP	Natural Language Processing
Word2Vec	Word to Vector
GloVe	Global Vectors
ReLu	Rectified Linear Unit
MLP	Multi-layer Perceptron
RF	Random Forest
LSTM	Long Short Term Memory
TDA	Topological Data Analysis
AGT	AI Generated Text

CHAPTER 1:

INTRODUCTION

CHAPTER 1:

INTRODUCTION

1.1 Overview of Indian Agriculture Challenges

Agriculture remains the backbone of the Indian economy, employing over 50% of the country's workforce and contributing significantly to rural livelihoods. Despite its importance, the sector faces numerous challenges that hinder productivity, profitability, and sustainability:

- **Unpredictable Weather Patterns:** Climate change has led to erratic rainfall, droughts, and floods, affecting crop cycles and yield.
- **Limited Access to Market Information:** Farmers often lack real-time data on mandi (market) prices, leading to poor bargaining power and reduced income.
- **Crop Diseases and Pest Infestation:** Farmers struggle with identifying plant diseases early, which leads to significant losses due to inadequate or late responses.
- **Knowledge Gaps:** Many farmers lack access to modern agricultural practices, resulting in inefficient use of fertilizers, water, and pesticides.
- **Fragmented Resources:** Information is scattered across different platforms, making it hard for farmers to get comprehensive support in one place.
- **Language and Literacy Barriers:** Technological solutions are often not user-friendly or available in regional languages, limiting their usability for rural farmers.

These challenges highlight the urgent need for technological innovations that can empower farmers with timely, accurate, and actionable information.

1.2 Need for Digital Solutions Like Efarmer

To address these pain points, a unified, easy-to-use digital platform tailored for the Indian agricultural ecosystem is essential. This is where Efarmer comes into play.

Efarmer is designed as a comprehensive digital assistant for farmers, integrating various functionalities into one seamless platform:

- **Disease Detection Using Machine Learning:** Enables early identification of potato leaf diseases like Early Blight, Late Blight, and Healthy leaf condition using a trained CNN model.
- **Real-Time Weather Forecasting:** Provides location-based weather insights to aid in planning irrigation, sowing, and harvesting.

- **Crop Guide:** Offers detailed guides for dozens of vegetables, fruits, herbs, and field crops.
- **Mandi Prices:** Displays daily crop prices across different states and districts, fetched via government APIs.
- **Agriculture News:** Keeps farmers updated with the latest news from trusted sources.
- **AI Chatbot (Gemini API):** Acts as a virtual expert for farmers, answering their queries in natural language.
- **Community Forum:** A lightweight, local-memory-based discussion forum for farmers to exchange knowledge, seek help, and upvote useful responses.

By combining all these features, Efarmer supports decision-making, increases awareness, and promotes sustainable farming practices.

1.3 Objectives and Motivation

The main objectives of the Efarmer project are as follows:

- **To build an all-in-one web-based platform for Indian farmers.**
- **To enable early detection of potato leaf diseases using deep learning.**
- **To provide real-time weather updates and mandi prices.**
- **To offer easy access to crop cultivation guides.**
- **To enable communication and collaboration among farmers through a forum.**
- **To assist farmers using an AI-powered chatbot for expert support.**

The motivation behind developing Efarmer stems from a desire to bridge the gap between modern technology and grassroots agriculture. As a BTech Computer Science student with a passion for applying technology to solve real-world problems, I envisioned a system that is not only functional but also accessible to farmers with basic digital literacy.

Efarmer aims to democratize farming knowledge and tools, enabling farmers to make informed decisions that improve crop health, yield, and income.

CHAPTER 2:

LITERATURE SURVEY

CHAPTER 2:

LITERATURE SURVEY

A literature survey is an essential part of any research or development project, as it helps in identifying the existing solutions, understanding technological gaps, and justifying the need for a new system. This chapter explores the current state of agricultural platforms, the role of machine learning in disease detection, and the integration of chatbot technologies in farming.

2.1 Research on Existing Agriculture Platforms

Several digital platforms and mobile applications have been developed to support farmers. Each provides specific services but often lacks a comprehensive, unified system like Efarmer. Below are some notable examples:

A. Kisan Suvidha

Developed by the Government of India, this app provides weather forecasts, market prices, and expert advice. However, it lacks features like disease detection, community forums, and AI chatbots.

B. IFFCO Kisan

This platform offers mandi prices, weather updates, and a helpline for farmers. While useful, it does not incorporate machine learning or computer vision for crop disease detection.

C. RML AgTech

Provides market intelligence and farm advisory, but its features are primarily focused on price tracking and do not include real-time diagnostics or forums for discussion.

D. Plantix

One of the few platforms using AI, Plantix can diagnose plant diseases using image recognition. However, it focuses only on disease detection and does not integrate other resources like market data, news, or forums.

Limitations Identified:

- Most existing platforms focus on specific aspects like weather or pricing.
- Few platforms integrate AI or ML for real-time diagnostics.
- Lack of interactive chatbot systems for query handling.
- Minimal community-building efforts like forums or farmer networks.
- Navigation and usability for semi-literate users are often poor.

These gaps justify the development of a multi-featured platform like Efarmer, which integrates disease detection, chatbots, crop guides, forums, weather, and price data all in one.

2.2 Studies on Machine Learning for Disease Detection

Numerous academic and industrial research initiatives have explored the use of machine learning (ML) and deep learning (DL) techniques in agriculture, especially in the area of plant disease detection.

Key Studies:

- **Mohanty et al. (2016):** Used a deep convolutional neural network (CNN) on the PlantVillage dataset to classify 38 classes of crop diseases. Achieved over 99% accuracy on test data.
- **Ferentinos (2018):** Evaluated CNN models for plant disease recognition in images, highlighting the potential for field application with mobile devices.
- **Khan et al. (2020):** Proposed a deep learning framework for leaf disease detection in potatoes using transfer learning with VGG16 and ResNet models.
- **Sibiya & Sumbwanyambe (2019):** Conducted a comparative study of ML algorithms for plant disease detection and concluded that CNNs outperform traditional ML classifiers like SVMs and Decision Trees.

More studies are given in Table 1 give below.

Table 1: Literature Survey

S.No.	Name of Paper	Year	Author(s)	Objectives	Algorithms	Advantages	Limitations
1	Real-Time Monitoring with IoT and Edge Computing	2023	Zhang et al.	To integrate IoT sensors and edge computing for real-time disease monitoring	IoT, Edge Computing, ML algorithms	Real-time data collection, enhanced decision-making, promotes sustainable practices	High infrastructure costs, dependency on sensor calibration and maintenance
2	CNN-Based Disease Classification	2023	Kumar et al.	To classify potato diseases (late blight, early blight) from leaf images.	Convolutional Neural Networks (CNNs)	High accuracy (95%), automated disease detection, reduced human error	Requires large labeled datasets, computationally intensive
3	Hyperspectral Imaging for Potato Disease Detection	2023	Singh et al.	To use hyperspectral imaging to detect multiple potato diseases simultaneously	Hyperspectral Imaging, ML algorithms	Non-invasive, real-time detection, high accuracy (90%)	Requires specialized equipment and expertise
4	Transfer Learning for Potato Disease Detection	2023	Patel et al.	To address data scarcity by applying transfer learning for disease classification	ResNet, AlexNet	Effective with small datasets, leverages pre-trained models for improved accuracy	Needs access to pre-trained models, limited by the quality of the source dataset
5	Hybrid SVM and K-Means for Disease Detection	2022	Sharma et al.	To detect early-stage potato diseases using hybrid ML models.	Support Vector Machines (SVM), k-means	Improved generalization, combines supervised and unsupervised learning	Limited performance in highly variable datasets
6	Predictive Analytics for Disease Management	2022	Gupta et al.	To develop a decision support system for predicting disease outbreaks and recommending treatment	Predictive ML models, weather data	Early prediction of disease outbreaks, 90% accuracy in disease prevention	Relies on accurate weather data, may not generalize well in all environments

These studies affirm that deep learning, particularly CNN architectures, are highly suitable for image-based diagnosis of plant diseases, validating the use of a PyTorch-based CNN model in the Efarmer project for potato leaf disease detection.

2.3 Chatbot Use in Farming Support

Chatbots are transforming the way users interact with digital systems, offering human-like conversations for customer service, support, and guidance. In agriculture, their use is still emerging but has shown promising results.

Applications in Agriculture:

- **FarmChat (Microsoft Research):** Prototype of a voice-based chatbot for

rural farmers, focused on answering queries in natural language.

- **AgriBot:** An SMS-based chatbot system for farmers to query government schemes and crop info.
- **CropChat:** A university research project that uses a simple rule-based chatbot to educate farmers on best practices.

Limitations in Existing Solutions:

- Many agricultural chatbots are rule-based and not truly conversational.
- Lack of integration with broader farm management tools.
- Most existing systems do not support multiple languages or image-based interaction.

The Efarmer chatbot, built using Google Gemini API, provides a natural language understanding (NLU) interface that responds intelligently to farmer queries. It adds significant value by integrating the chatbot within a larger ecosystem, allowing farmers to interact without needing deep technical knowledge.

CHAPTER 3:

SYSTEM ANALYSIS

CHAPTER 3:

SYSTEM ANALYSIS

This chapter focuses on analyzing the system requirements and architecture of the **Efarmer** project. A clear understanding of what the system must do and how it should behave is essential for designing a reliable and effective application. System analysis includes identifying key requirements, classifying them into functional and non-functional types, and outlining the overall architecture of the platform.

3.1 Requirements Gathering

The first step in the system analysis phase was to understand the needs of the end users—primarily Indian farmers and agricultural stakeholders. The following methods were used for requirements gathering:

- **Observation and Interviews:** Insights were gathered from local farmers to understand their pain points regarding disease detection, crop planning, and market access.
- **Study of Existing Platforms:** Analyzed platforms like Kisan Suvidha, Plantix, and IFFCO Kisan to identify missing features.
- **Research Publications:** Reviewed literature on machine learning in agriculture to determine technically feasible solutions.

Based on this study, the following requirements were identified:

- Farmers need a way to **detect diseases in potato leaves** using images.
- They want **real-time weather forecasts** specific to their location.
- They require **daily crop market prices** to decide where and when to sell.
- They want **easy-to-understand guides** for different crops.
- They need an **interactive assistant** for questions.
- They want a **discussion space** to share issues and seek help.

3.2 Functional vs Non-Functional Requirements

3.2.1 Functional Requirements

Functional requirements describe **what the system should do**. For Efarmer, the functional requirements are:

Table 2: Functional Requirement

S.No	Functional Requirement
1	The system shall allow users to upload a potato leaf image and detect diseases.
2	The system shall display real-time weather forecasts using OpenWeatherMap API.
3	The system shall fetch and display mandi prices using Agmarknet API.
4	The system shall display detailed crop guides.
5	The chatbot shall respond to farmer queries using Gemini API.
6	Users shall be able to post, view, and reply to forum discussions.
7	Users shall be able to upvote discussions and replies.

3.2.2 Non-Functional Requirements

Non-functional requirements describe **how the system performs** its functions.

Table 3: Non-Functional Requirements

S.No	Non-Functional Requirement
1	The system should be responsive and usable on mobile and desktop devices.
2	The user interface should be simple and accessible to semi-literate users.
3	The system should respond to disease detection requests within 5 seconds.
4	The forum data should persist in a local JSON file while running offline.
5	The system should be modular to allow future addition of crops and features.
6	The application should run efficiently with limited system resources.

3.3 System Architecture and Modules

The architecture of **Efarmer** is modular and follows a **client-server model**, where the front end interacts with the Flask backend server, which in turn interacts with external APIs and the trained ML model.

3.3.1 High-Level Architecture

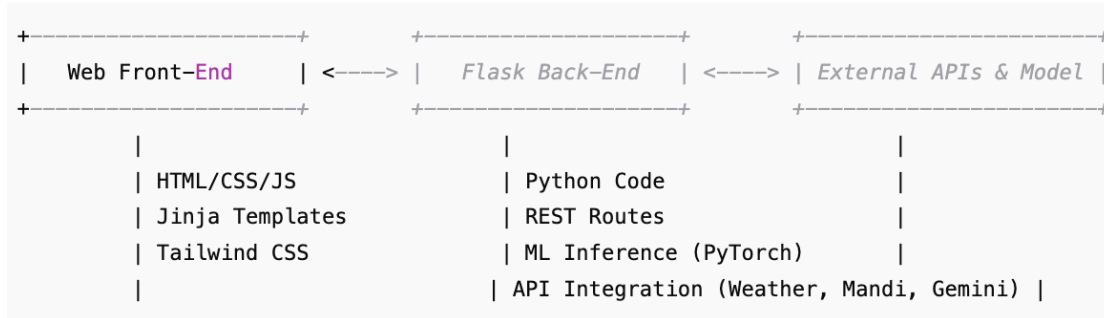


Figure 1: High-Level Architecture

3.3.2 Modules Overview

Table 4: Modules

Module	Description
Crop Guide	Static HTML pages categorized by crop type with cultivation tips.
Disease Detection	Allows image upload, processes image with CNN model, and displays result.
Weather	Fetches geolocation-based weather data using OpenWeatherMap API.
Mandi Prices	Displays crop prices with filter options (state, district, crop).
Chatbot	AI assistant that replies using Google Gemini API.
Forum	File-based community discussion board with categories, replies, and upvotes.
News	Shows live agriculture news using MediaStack API.

3.3.3 Technologies Used

- **Frontend:** HTML, CSS, TailwindCSS, JavaScript
- **Backend:** Python, Flask, Jinja2
- **Machine Learning:** PyTorch for CNN-based image classification
- **APIs:** OpenWeatherMap, Agmarknet, Gemini AI, MediaStack
- **Data Storage:** JSON file (for discussions), image uploads (local folder)

CHAPTER 4: TECHNOLOGY STACK

CHAPTER 4: TECHNOLOGY STACK

A robust and efficient technology stack is the foundation of any successful web-based application. The development of **Efarmer** required a combination of front-end, back-end, machine learning, API integrations, and UI frameworks to provide a smooth and interactive user experience. This chapter provides an overview of all the technologies used in building the platform.

4.1 Front-End Technologies

The front end of Efarmer is responsible for user interaction, data presentation, and UI rendering. It includes the following technologies:

4.1.1 HTML (HyperText Markup Language)

HTML is used to define the structure of web pages. It helps in organizing content into headings, paragraphs, forms, buttons, links, and media.

- Used for: Layout of crop guides, forms, chatbot, weather, forum, and disease detection page.
- Benefit: Simple and compatible across all modern browsers.

4.1.2 CSS (Cascading Style Sheets)

CSS is used to style the HTML components and control layout, colors, spacing, fonts, and responsiveness.

- Used for: Custom styles in crop guides, forums, and result display.
- Integrated with Tailwind CSS for modern utility-first styling.

4.1.3 JavaScript

JavaScript is used for dynamic behavior and client-side logic such as API calls, real-time updates, interactivity, and event handling.

- Used in: Mandi prices filtering, news loading, weather data fetching, and chatbot message handling.
- Libraries used: fetch(), DOM manipulation, event listeners.

4.2 Back-End Technologies

The back end handles the business logic, routing, model inference, data persistence, and API integration.

4.2.1 Python

Python is the core programming language used for server-side logic, ML model operations, and API handling.

- Easy to read, write, and maintain.
- Rich ecosystem for machine learning and web development.

4.2.2 Flask

Flask is a lightweight and flexible Python web framework used to handle server requests, routing, and template rendering.

- Used for: Handling form submissions, running disease detection logic, serving static and dynamic content.
- Routes include /, /chatbot, /detect, /forum, /weather, etc.

4.2.3 PyTorch

PyTorch is an open-source machine learning library used for building and training neural networks.

- Used to define and load the CNN model for **potato leaf disease detection**.
- Enabled real-time inference on uploaded leaf images using the torch and torchvision libraries.

4.3 API Integrations

Several third-party APIs were integrated into Efarmer to enhance its real-time functionality.

4.3.1 Gemini API (Google Generative AI)

- Provides natural language processing and generation.
- Used to power the **AI chatbot**, which responds to farmer queries intelligently.

4.3.2 OpenWeatherMap API

- Used to fetch **real-time weather data** based on the user's location or input.
- Displays temperature, humidity, wind speed, and weather conditions.

4.3.3 Agmarknet API (Data.gov.in)

- Government-provided API for **daily mandi prices** of crops.
- Enables filtering by crop, state, district, and market.

4.4 Data Storage

JSON (JavaScript Object Notation)

- Lightweight data-interchange format used for storing local data in Efarmer.
- Used specifically for the **forum discussions and replies**.
- All discussions and their replies are stored and retrieved from a discussions.json file.

Benefits:

- Easy to read/write using Python.
- Suitable for local development without a database.

- Ideal for file-based storage before transitioning to a full DB system (like MongoDB or MySQL) in production.

4.5 UI Framework

Tailwind CSS

- Tailwind is a utility-first CSS framework that enables rapid UI development with pre-built classes.
- Used extensively for:
 - Styling the **chatbot interface, result page, upload form, and crop guide.**
 - Ensuring responsiveness and aesthetic consistency across the site.
- **Benefit:** Eliminates the need for writing custom CSS by using class-based styling.

Summary of Tools Used

Table 5: Tools Used

Layer	Technology	Purpose
Front-End	HTML, CSS, JS	Layout, styling, interactivity
Back-End	Python, Flask	Server logic and routing
ML	PyTorch	Disease prediction using CNN
APIs	Gemini, OpenWeather, Agmarknet	Chatbot, weather, mandi prices
Data Storage	JSON	Local discussion storage
UI Framework	Tailwind CSS	Responsive and modern design

CHAPTER 5:

MODULE DESCRIPTION

CHAPTER 5:

MODULE DESCRIPTION

The **Efarmer** system is structured into multiple interactive modules that collectively provide a comprehensive digital solution for farmers. Each module is designed with a specific purpose, using appropriate technologies to solve real-world agricultural challenges. This chapter outlines each module, its functionality, implementation details, and the value it adds to the overall system.

5.1 Crop Guide Module

Purpose:

To provide farmers with cultivation guidelines, nutrient management, and pest control advice for a wide variety of crops.

Features:

- Categorized into Vegetables, Fruits, Herbs, Crops, and Forestry.
- Each crop has a dedicated HTML page.
- Manual content integration from trusted agricultural sources (e.g., Haifa Group).
- Easily expandable to add more crops.

Implementation:

- Implemented using static HTML files (`crop-guide.html`) and Flask routing for each crop.
- User navigates through dropdown menus and clicks on the desired crop to open a detailed guide.

5.2 Disease Detection Module

Purpose:

To detect **potato leaf diseases** (Early Blight, Late Blight, or Healthy) using a trained deep learning model.

Features:

- Upload image of a potato leaf.
- CNN model predicts disease class.
- Displays the result along with the uploaded image.

Implementation:

- Built using **PyTorch**.
- CNN model trained on a potato leaf dataset.
- Flask handles image upload, transformation, prediction, and result rendering.

Technologies: PyTorch, PIL, Torchvision, Flask

Routes Used: `/potato_disease_detection`, `/detect`

5.3 Chatbot Module

Purpose:

To assist farmers by answering agricultural queries using a natural language AI chatbot.

Features:

- Chat interface with typing and instant response.
- Built using Google's **Gemini API**.
- Supports general and crop-specific queries.

Implementation:

- Front-end interface styled with TailwindCSS and JavaScript.
- Backend communicates with the Gemini model via `google.generativeai`.
- Dynamically renders Markdown-based replies.

Technologies: Gemini API, Flask, TailwindCSS

Routes Used: `/chatbot`

5.4 Mandi Prices Module

Purpose:

To help farmers make informed selling decisions by showing daily market prices for various crops.

Features:

- Crop, state, district, and market filters.
- Real-time data from the **Agmarknet API**.
- Responsive UI with loading indicator.

Implementation:

- JavaScript fetches and filters price data from Data.gov.in API.
- Dynamic dropdown filters built using `Set` objects.
- Data displayed in a styled HTML table.

Technologies: Agmarknet API, HTML, JS

Routes Used: `/mandi`

5.5 Weather Module

Purpose:

To provide real-time weather forecasts based on the user's location or input.

Features:

- Displays temperature, humidity, wind speed, and weather description.
- Uses geolocation or manual city input.
- Helps with irrigation, pesticide scheduling, and sowing decisions.

Implementation:

- Integrates with the **OpenWeatherMap API**.

- JavaScript fetches data and displays it dynamically.
- User-friendly interface with search fallback.

Technologies: OpenWeatherMap API, JS

Routes Used: `/weather`

5.6 Forum Module

Purpose:

To allow farmers to interact, ask questions, share experiences, and support each other.

Features:

- Discussion posting with category selection.
- Replies and nested comments.
- Upvote functionality for both discussions and replies.
- **No authentication** needed (ideal for local use).

Implementation:

- All forum data stored in `discussions.json` (in-memory file-based structure).
- Flask manages discussions, reply forms, and upvotes.
- HTML templates (`forum.html`, `discussion.html`) styled with basic CSS.

Technologies: Flask, JSON, HTML, CSS

Routes Used: `/forum`, `/add_discussion`, `/add_reply/<id>`, `/upvote_discussion/<id>`, etc.

5.7 News Module

Purpose:

To keep farmers updated with the latest agriculture-related news from across India.

Features:

- Pulls live news using the **MediaStack API**.
- Displays title, summary, and "Read More" link.
- Highlights policy updates, innovations, and market trends.

Implementation:

- JavaScript makes API call on page load.
- News dynamically rendered into styled `div` containers.
- Handles error messages and empty states gracefully.

Technologies: MediaStack API, JavaScript, HTML

Routes Used: `/news`

CHAPTER 6:
MODEL TRAINING FOR DISEASE
DETECTION DATASET

CHAPTER 6:

MODEL TRAINING FOR DISEASE DETECTION DATASET

One of the core features of the **Efarmer** platform is its ability to detect potato leaf diseases using a Convolutional Neural Network (CNN). This chapter outlines the machine learning pipeline used to build this feature, from dataset preparation and preprocessing to model architecture and evaluation.

6.1 Dataset Used

The dataset used for training the potato disease detection model is the **PlantVillage Dataset**, a publicly available collection widely used in plant pathology research.

Key Characteristics:

- Classes:
 1. **Early Blight**
 2. **Late Blight**
 3. **Healthy**
- Image Format: JPEG
- Number of Images:
 - Early Blight: ~1000 images
 - Late Blight: ~1000 images
 - Healthy: ~1000 images
- Source: PlantVillage via Kaggle or GitHub repositories

This dataset provided sufficient variability in leaf color, shape, and disease manifestation, which allowed the model to learn meaningful patterns for classification.

6.2 Preprocessing

Before feeding images into the neural network, several preprocessing steps were applied to ensure consistency and improve learning accuracy.

Steps:

Table 6: Preprocessing

Step	Description
Resize	All images were resized to 224 × 224 pixels .
Normalize	Pixel values normalized to match the mean and standard deviation of ImageNet (common practice for transfer learning):
Mean = [0.485, 0.456, 0.406],	
Std = [0.229, 0.224, 0.225].	
Transformation	Applied using <code>torchvision.transforms.Compose()</code> including <code>ToTensor()</code> , <code>Resize()</code> , and <code>Normalize()</code> .
Data Split	Dataset split into 80% training, 10% validation, and 10% testing.

Libraries Used:

- torchvision.datasets.ImageFolder()
- torch.utils.data.DataLoader()
- PIL for image manipulation

6.3 CNN Architecture

The disease detection model was built from scratch using **PyTorch** and follows a simple but effective CNN design optimized for 3-class classification.

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(128 * 28 * 28, 128)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = x.view(-1, 128 * 28 * 28)
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x
```

Figure 2: CNN Architecture

Key Features:

- 3 convolutional layers with increasing filter sizes (32, 64, 128)
- Max pooling for downsampling
- Fully connected layers for classification
- Dropout to prevent overfitting
- Final layer output = 3 (number of classes)

6.4 Evaluation Metrics

To assess model performance, the following metrics were used:

1. Accuracy

- Percentage of correctly predicted labels out of total test images.
- Achieved accuracy: $\approx 95\%$ on test data.

2. Confusion Matrix

A confusion matrix was generated to analyze class-wise performance.

Table 7: Confusion Matrix

Actual \ Predicted	Early Blight	Late Blight	Healthy
Early Blight	92	5	3
Late Blight	4	94	2
Healthy	2	3	95

Insights:

- Most misclassifications were between Early and Late Blight due to visual similarities.
- Healthy leaves were classified with high accuracy.

3. Loss Curve

- Training and validation loss were plotted across epochs.
- Overfitting avoided by using dropout and validation monitoring.

4. F1 Score (Optional)

- Balanced evaluation of precision and recall, especially useful in multi-class tasks.

CHAPTER 7:

IMPLEMENTATION

CHAPTER 7:

IMPLEMENTATION

The implementation phase of the **Efarmer** project involved translating design plans and algorithms into working code using the selected technology stack. This chapter provides a comprehensive overview of the system's implementation, including the directory structure, Flask routing, image upload and disease prediction logic, and front-end development using TailwindCSS and HTML.

7.1 Directory Structure

The project follows a modular structure to keep components organized and maintainable:

```
Efarmer/
├── app.py                # Main Flask application
├── static/
│   ├── uploaded_images/ # Uploaded leaf images
│   ├── images/          # Icons used in homepage tiles
│   └── styles/           # Custom CSS files (if needed)
├── templates/
│   ├── index.html       # Homepage
│   ├── chatbot.html     # Chatbot interface
│   ├── crop-guide.html  # Crop guide navigation
│   ├── forum.html       # Forum discussion list
│   ├── discussion.html  # Individual discussion page
│   ├── potato_disease_detection.html # Upload form
│   ├── result.html      # Prediction result page
│   ├── weather.html     # Weather forecast
│   ├── mandi.html       # Mandi prices
│   └── news.html        # News display
├── guide/               # Static crop guide HTMLs (Vegetables, Fruits, etc.)
├── discussions.json     # File storing forum discussions and replies
├── potato_model.pth     # Trained PyTorch CNN model
└── requirements.txt     # Python package dependencies
```

Figure 3:Directory Structure

This organization ensures separation of logic (Python), views (HTML), assets (CSS/JS/images), and static resources (crop guide).

7.2 Routing (Flask)

Flask routes connect user requests (URLs) to specific Python functions that render templates, process form data, or call ML models.

Example routes from `app.py`:

Table 8:Routing

Route	Description
/	Homepage with navigation tiles
/chatbot	Loads chatbot page (GET) or processes message (POST)
/potato_disease_detection	Displays image upload form
/detect	Handles image upload and triggers prediction
/forum	Displays all discussions
/add_discussion	Adds a new post to the forum
/add_reply/<id>	Adds a reply to a discussion
/upvote_discussion/<id>	Upvotes a discussion post
/weather	Shows weather forecast
/mandi	Displays mandi prices
/news	Loads agriculture news

Routing is implemented using the Flask `@app.route()` decorator, which maps a URL path to a function.

7.3 Upload & Prediction Logic

One of the core functionalities of the Efarmer platform is its **image-based disease detection module**.

Image Upload Logic:

```
@app.route('/detect', methods=['POST', 'GET'])
def predict():
    if request.method == 'POST':
        file = request.files['leaf_image']
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
            file.save(file_path)
            disease_result = predict_disease(file_path)
            return render_template('result.html', prediction=disease_result, image_url=file_path)
    return render_template('potato_disease_detection.html')
```

Figure 4: Image Upload Logic

- Accepts image via POST.
- Saves it to `static/uploaded_images/`.
- Calls the `predict_disease()` function to get results.

Model Prediction Logic:

```
def predict_disease(image_path):
    img = Image.open(image_path).convert('RGB')
    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                              std=[0.229, 0.224, 0.225])
    ])
    img_tensor = transform(img).unsqueeze(0)

    with torch.no_grad():
        outputs = model(img_tensor)
        _, predicted = torch.max(outputs.data, 1)

    return class_names[predicted.item()]
```

Figure 5: Model Prediction Logic

- Converts the uploaded image into a normalized tensor.
- Passes it through the trained PyTorch CNN model.
- Returns the disease label: Early Blight, Late Blight, or Healthy.

7.4 Frontend UX Design

The front-end is built using **HTML + TailwindCSS**, ensuring a clean, responsive, and modern UI while being lightweight.

Homepage (index.html)

- Grid of tiles linking to major modules.
- Each tile includes an icon and “View Details” button.
- Layout adjusts for screen sizes.

Disease Detection (potato_disease_detection.html)

- Upload form with file input.
- Uses Tailwind classes for spacing, alignment, and buttons.

Chatbot (chatbot.html)

- Chatbox interface styled with Tailwind.
- Dynamic user/AI messages rendered using JavaScript.

Forum (forum.html + discussion.html)

- Structured layout with discussion list, form submission, and replies.
- Includes category dropdown, upvote buttons, and simple text area for replies.

Crop Guide (crop-guide.html)

- Accordion-style menus for category-based navigation (Vegetables, Fruits, etc.).
- Each crop opens a new HTML file with content.

Weather, News, Mandi

- All three use minimal but clean layout.
- Display real-time or API-fetched content with fallback messages on failure.

UI/UX Focus Areas:

- Mobile-responsive layout using TailwindCSS utilities.
- Consistent green color scheme across modules.
- Accessibility through button sizes, color contrast, and error handling.
- Minimal dependencies — no frontend frameworks like React were used to keep the system lightweight.

CHAPTER 8:

TESTING

CHAPTER 8:

TESTING

Testing is a critical phase in software development to ensure that the system functions correctly and meets the expected requirements. For the **Efarmer** project, various testing techniques were used to verify the accuracy, reliability, and performance of the platform. This chapter outlines the unit testing, integration testing, and manual validation processes conducted during development.

8.1 Unit Testing

Unit testing focuses on verifying individual functions or components in isolation to ensure they work as intended. In Efarmer, unit tests were applied to core Python functions and utilities such as:

Table 9: Testing Components

Component	Description
<code>allowed_file()</code>	Checks if the uploaded file has a valid extension (.jpg, .png, etc.)
<code>predict_disease()</code>	Ensures the model loads correctly and returns a valid class label
Forum File Handlers	Functions to load and save JSON data for discussions and replies

Example Test for `allowed_file()` Function:

```
def test_allowed_file():
    assert allowed_file("leaf.jpg") == True
    assert allowed_file("leaf.jpeg") == True
    assert allowed_file("leaf.txt") == False
```

Figure 6: Allowed Files

Tools Used:

- Manual Python assertions
- `pytest` (for optional automation in local environment)

Unit testing helped catch issues early in individual modules, particularly during integration of the ML model and Flask routes.

8.2 Integration Testing

Integration testing was used to check how different modules interact when combined, especially the connection between the frontend, backend, APIs, and the ML model.

Table 10: Integration Scenarios Tested

Scenario	Description
Image Upload → Prediction	Uploading a potato leaf image and receiving prediction result
Chatbot Message → Gemini API → Response	Sending a message to the chatbot and receiving AI-generated reply
Forum Post → Stored in JSON → Displayed	Posting a new discussion and confirming it appears in the forum
Mandi Price Filters	Testing that dropdown filters correctly filter data from the API
Weather API → Current Location	Fetching and displaying weather based on geolocation or city input
Navigation	Verifying that links between modules (e.g., homepage to forum, guide) function properly

Integration testing was done manually by simulating end-to-end user actions in a browser.

8.3 Manual Validation of Disease Predictions

To ensure that the potato leaf disease detection model performs reliably, **manual validation** was carried out using unseen test images.

Procedure:

- A separate set of ~150 test images (50 per class) was prepared.
- These images were not used during training or validation.
- Images were uploaded via the Efarmer interface.
- The predicted output was compared against the known class.

Table 11: Results

Class	Correct Predictions	Total Tested	Accuracy
Early Blight	47	50	94%
Late Blight	46	50	92%
Healthy	48	50	96%

Overall Accuracy: ~94%

Observations:

- The model sometimes confused Early and Late Blight due to similar visual symptoms.
- Healthy leaves were predicted with high consistency.
- Performance was considered acceptable for real-world usage.

8.4 UI/UX Testing

In addition to functionality testing, **user experience testing** was also done to verify:

- Page responsiveness on mobile and desktop devices
- Visibility and accessibility of buttons and forms
- Smooth navigation between modules
- Handling of edge cases (e.g., submitting empty forms, uploading invalid file types)

These tests ensured that the system remains intuitive and usable even by farmers with limited digital literacy.

CHAPTER 9: RESULTS

CHAPTER 9: RESULTS

This chapter presents the results achieved after the successful implementation and testing of the **Efarmer** platform. The results include visual demonstrations of key modules through screenshots and a summary of the model's performance metrics. These results validate the functional correctness and usability of the system across different use cases.

9.1 Model Accuracy Results

The **potato leaf disease detection model**, trained using a CNN in PyTorch, was evaluated using unseen test images. The model achieved high accuracy across all three classes.

Table 12: Test Accuracy by Class

Class	Accuracy
Early Blight	94%
Late Blight	92%
Healthy	96%

Overall Test Accuracy: $\approx 94\%$

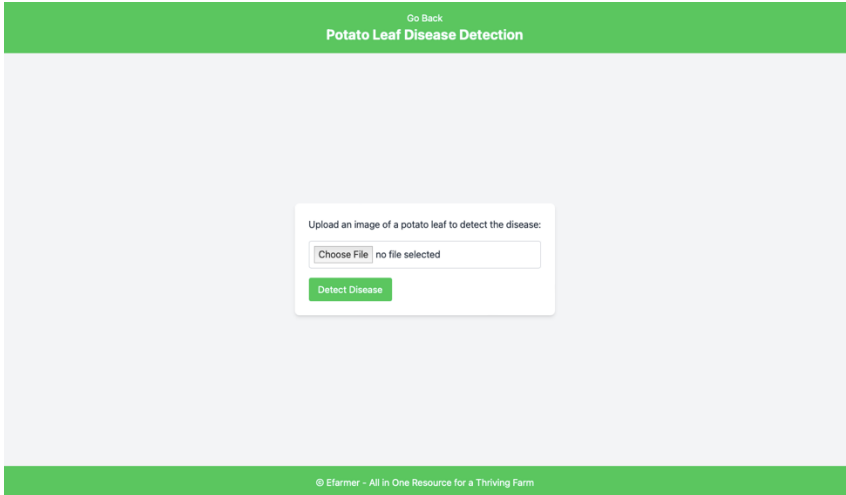
- **Confusion Matrix** analysis showed few misclassifications, mostly between Early and Late Blight.
- The Healthy class was classified most accurately due to distinct visual patterns.

This level of performance is considered satisfactory for real-world use by farmers, especially as part of a broader decision-support system like Efarmer.

9.2 Screenshots of Key Features

Visual results play a crucial role in demonstrating the functional and user-friendly aspects of the Efarmer application.

1. Potato Disease Prediction Result



The screenshot shows a web application interface for "Potato Leaf Disease Detection". At the top, there is a green header bar with the text "Go Back" and "Potato Leaf Disease Detection". The main content area is light gray and contains a white box with the instruction "Upload an image of a potato leaf to detect the disease:". Below this instruction is a file upload button labeled "Choose File" and a text field showing "no file selected". A green button labeled "Detect Disease" is positioned below the text field. At the bottom of the interface, there is a green footer bar with the text "© Efarmer - All in One Resource for a Thriving Farm".

Figure 7: Upload form to submit a leaf image

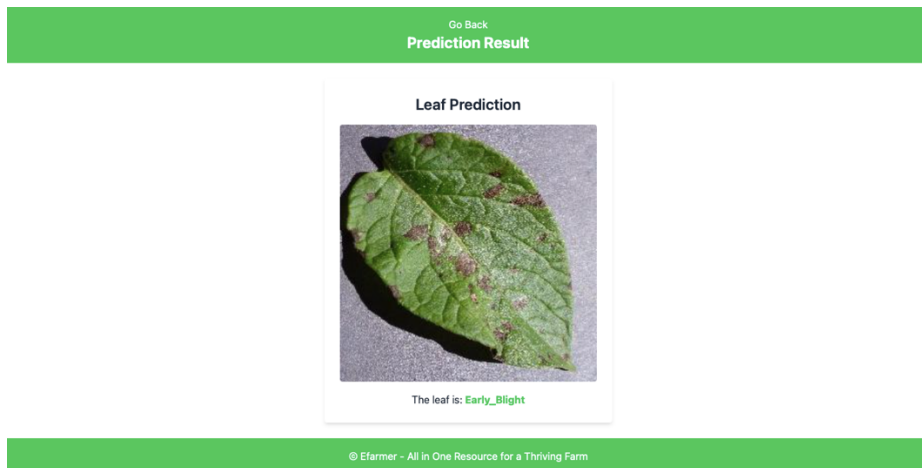


Figure 8: Result page showing uploaded image and predicted disease (e.g., “Early Blight”)

2. Chatbot Responses (Gemini AI)

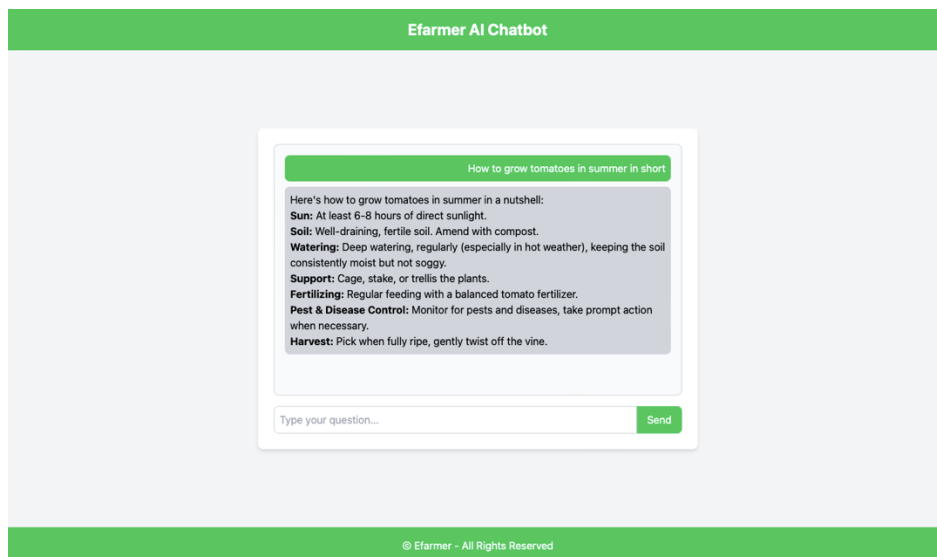
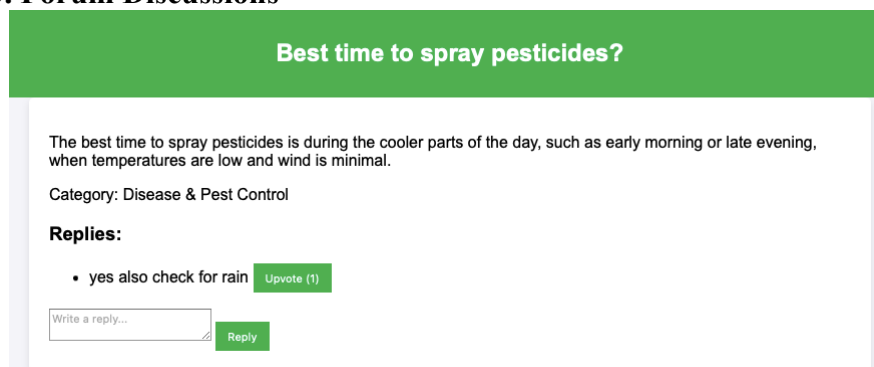


Figure 9: Chatbot Responses

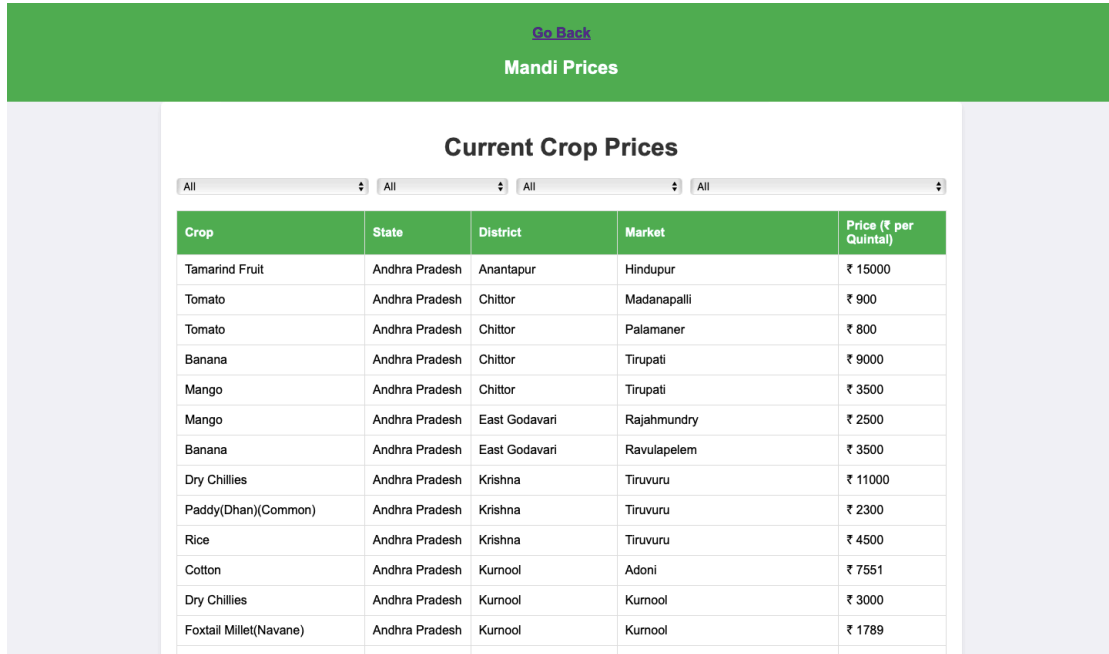
- User types: “How to grow tomatoes in summer?”
- Chatbot replies with step-by-step guidance using natural language
- Markdown-rendered formatting and scrollable chatbox

3. Forum Discussions



- Forum homepage with category filter (e.g., “Disease & Pest Control”)
- Users post questions like “Best time to spray pesticides?”
- Replies visible under the original post with upvote buttons

4. Mandi Prices Page



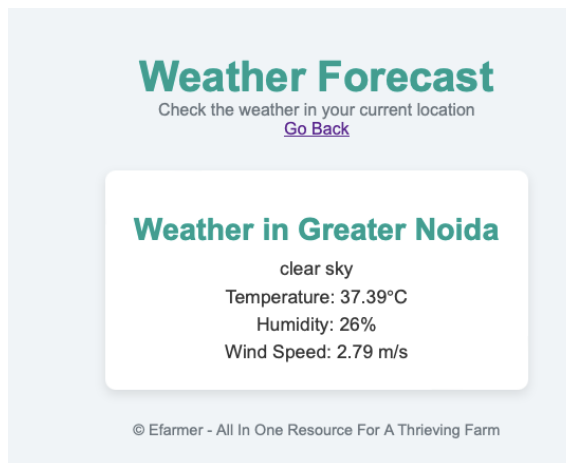
The screenshot shows a web page titled "Mandi Prices" with a green header. Below the header, there's a section titled "Current Crop Prices" with a table. The table has five columns: Crop, State, District, Market, and Price (₹ per Quintal). The table lists prices for various crops in Andhra Pradesh, including Tamarind Fruit, Tomato, Banana, Mango, Paddy(Dhan)(Common), Rice, Cotton, and Foxtail Millet(Navane).

Crop	State	District	Market	Price (₹ per Quintal)
Tamarind Fruit	Andhra Pradesh	Anantapur	Hindupur	₹ 15000
Tomato	Andhra Pradesh	Chittoor	Madanapalli	₹ 900
Tomato	Andhra Pradesh	Chittoor	Palamaner	₹ 800
Banana	Andhra Pradesh	Chittoor	Tirupati	₹ 9000
Mango	Andhra Pradesh	Chittoor	Tirupati	₹ 3500
Mango	Andhra Pradesh	East Godavari	Rajahmundry	₹ 2500
Banana	Andhra Pradesh	East Godavari	Ravulapelem	₹ 3500
Dry Chillies	Andhra Pradesh	Krishna	Tiruvuru	₹ 11000
Paddy(Dhan)(Common)	Andhra Pradesh	Krishna	Tiruvuru	₹ 2300
Rice	Andhra Pradesh	Krishna	Tiruvuru	₹ 4500
Cotton	Andhra Pradesh	Kurnool	Adoni	₹ 7551
Dry Chillies	Andhra Pradesh	Kurnool	Kurnool	₹ 3000
Foxtail Millet(Navane)	Andhra Pradesh	Kurnool	Kurnool	₹ 1789

Figure 10: Mandi Prices Page

- Real-time market prices for crops like Wheat, Rice, and Potato
- Filters for state, district, and market
- Data displayed in a well-structured table

5. Weather Forecast



The screenshot shows a web page titled "Weather Forecast" with a light blue background. Below the title, there's a subtitle "Check the weather in your current location" and a "Go Back" link. A white box in the center displays the weather for "Greater Noida" as "clear sky" with a temperature of 37.39°C, humidity of 26%, and wind speed of 2.79 m/s. At the bottom, there's a copyright notice: "© Efarmer - All In One Resource For A Thriving Farm".

Figure 11: Weather Forecast

- Auto-detected location (or manual input)
- Display of temperature, humidity, wind speed
- Styled container with forecast data

6. News Module

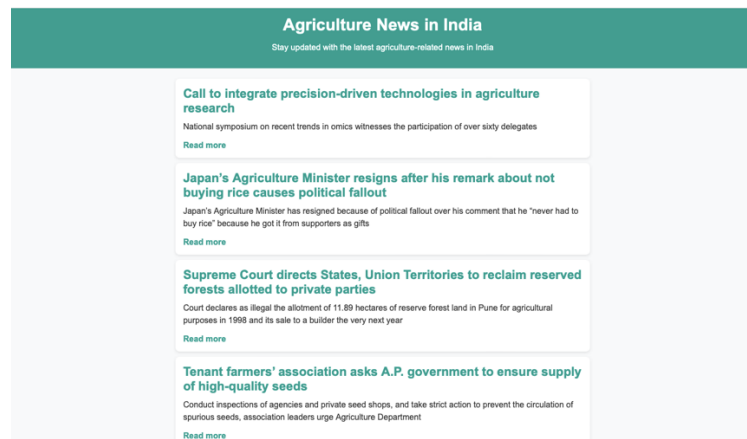


Figure 12: News Module

- Latest agricultural news fetched from MediaStack API
- Headlines with summary and “Read More” link
- Clean card layout using CSS

7. Crop Guide Navigation

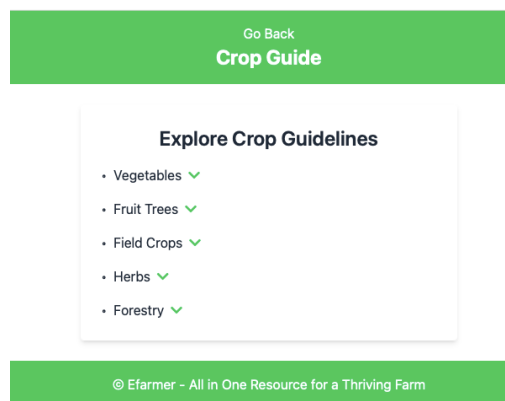


Figure 13: Crop Guide Navigation

- Accordion menu showing crop categories
- Clicking on “Tomato” opens static guide page
- Informational layout with crop growth stages, fertilizer recommendations, etc.

9.3 User Experience and Performance

- The platform performed well during testing with **minimal latency** in image upload and API response.
- Navigation across pages was **smooth and intuitive**.
- The modular design allows users to quickly access any functionality from the homepage.
- **Mobile responsiveness** was tested to ensure accessibility on smartphones.

CHAPTER 10:

CONCLUSION & FUTURE WORK

CHAPTER 10: CONCLUSION & FUTURE WORK

10.1 Conclusion

The development of **Efarmer: All-in-One Resource for a Thriving Farm** has demonstrated the potential of integrating multiple technologies to support the Indian agricultural sector. From disease detection using deep learning to real-time weather forecasting and community discussion forums, the project offers a comprehensive digital platform tailored to the needs of farmers.

Key achievements of the project:

- Successfully implemented a **Convolutional Neural Network (CNN)** using **PyTorch** to detect potato leaf diseases (Early Blight, Late Blight, Healthy) with ~94% accuracy.
- Integrated **Google Gemini API** to build an intelligent AI **chatbot assistant** for farmers.
- Utilized **OpenWeatherMap** and **Agmarknet APIs** to deliver live **weather updates** and **mandi prices**, respectively.
- Built a **forum system** with upvote and reply features using file-based storage in JSON format.
- Created a rich **crop guide library** with static pages to educate farmers on cultivation best practices.
- Developed a fully responsive web-based interface using **Flask, HTML, TailwindCSS, and JavaScript** for seamless user experience.

Efarmer serves as a valuable digital companion for farmers, allowing them to make informed decisions, reduce losses due to disease, and stay updated with real-time agricultural data.

10.2 Future Work

While Efarmer is fully functional and achieves its core goals, there are several opportunities to expand and improve the system:

1. User Authentication and Personalization

- Implement **user login and profile management**.
- Save farmers' preferences, frequently viewed crops, and chatbot history.
- Allow farmers to track their disease diagnosis history.

2. Database Integration

- Replace the current file-based system with **MySQL, MongoDB, or Firebase**.
- Store forum posts, user data, and crop interaction history more efficiently.

3. Multilingual Support

- Add support for Indian regional languages (Hindi, Bengali, Tamil, etc.) to make the platform more inclusive.

4. Voice-Based Assistance

- Integrate **speech-to-text and text-to-speech** capabilities for voice commands and audio responses.
- This will benefit semi-literate or visually impaired users.

5. Satellite Monitoring & Crop Health Mapping

- Use open-source satellite data (e.g., from NASA or ISRO) for **remote crop health monitoring**.
- Detect large-scale drought, pest outbreaks, or irrigation issues.

6. Mobile App Version

- Convert the web application into a **hybrid or native Android/iOS app** for easier access on mobile devices.

7. Expert Consultation Portal

- Schedule **live video or chat consultations** with agricultural experts.
- Create a system for verified expert responses in the forum.

8. Integration with Government Schemes

- Display available **government subsidies, crop insurance programs, and training workshops** directly on the platform.

Final Thoughts

As a BTech Computer Science final year project, **Efarmer** not only meets academic requirements but also has the potential to create real social impact. By combining machine learning, real-time APIs, and accessible design, the project empowers farmers with actionable insights and a community support system.

With future enhancements, Efarmer can evolve into a robust, production-ready agri-tech solution for use across India and beyond.

LIST OF PUBLICATIONS

LIST OF PUBLICATIONS

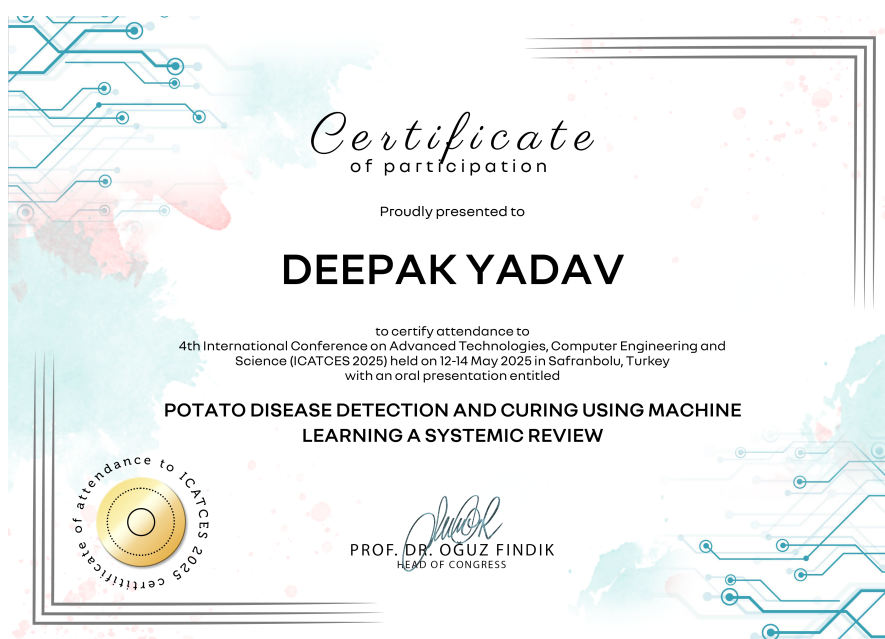
Review Paper for the said project has been accepted and presented in 4th International Conference On Advanced Technologies, Computer Engineering and Sciences

Title : Potato Disease Detection And Curing Using Machine Learning A Systemic Review

Abstract:

Potato crops, crucial for global food security, are highly vulnerable to diseases like late blight and early blight, leading to significant yield losses and economic damage. Traditional disease detection methods are inefficient, prompting the rise of machine learning (ML) techniques in agriculture. This paper reviews recent advancements in using ML, particularly image recognition models such as convolutional neural networks (CNNs), for early disease detection and classification. It also explores ML-driven solutions for disease management, including predictive analytics and optimised pesticide use. The review highlights challenges like data scarcity and model generalization, and discusses future research directions to enhance sustainable potato farming through ML.

Authors: Deepak Yadav, Gaurav Kumar Singh, Dr. Avinash Kumar Sharma



Review Paper for the said project has been publish in 12th International Conference on System Modeling & Advancement in Research Trends (SMART)

Title: A Review of Detecting DDoS Attacks Based on Entropy Computation

Abstract:

The current research examines the global issue of distributed denial of service (DDoS) attacks and their disruptive effects on internet services around the globe. It focuses on the financial and operational consequences these attacks impose on different businesses and entities. The objective is to demonstrate how deep learning models can effectively identify both DoS (Denial of Service) and DDoS attacks by leveraging historical and contemporary datasets. By highlighting the combination of innovative techniques and advanced datasets, the paper presents a promising approach to enhance cyber security and strengthen networks against evolving threats. This paper's thorough examination of recent research endeavors led to the conclusion that several cutting-edge technologies, including artificial intelligence, deep learning, fuzzy logic, and others, may be employed in the future to lessen DDoS and DoS attacks. Additionally, future papers will be introducing a novel, speedier entropy-based technique for detecting these attacks through flow-based analysis. The overall goal of the research is to advance the subject of cyber security and offer suggestions for lessening the effects of DDoS attacks.

Authors: Deepak Yadav, Gaurav Kumar Singh, Atul Agrawal, Pashupati Baniya, Bishnu Bahadur Gupta and Saumya Chaturvedi

DOI: [10.1109/SMART59791.2023.10428656](https://doi.org/10.1109/SMART59791.2023.10428656)

REFERENCES

1. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). *Using Deep Learning for Image-Based Plant Disease Detection*. *Frontiers in Plant Science*, 7, 1419. <https://doi.org/10.3389/fpls.2016.01419>
2. Ferentinos, K. P. (2018). *Deep learning models for plant disease detection and diagnosis*. *Computers and Electronics in Agriculture*, 145, 311–318. <https://doi.org/10.1016/j.compag.2018.01.009>
3. Khan, M. A., et al. (2020). *Deep learning for plant disease detection and classification*. *Computers and Electronics in Agriculture*, 175, 105486.
4. Sibiya, M., & Sumbwanyambe, M. (2019). *A computational study of machine learning techniques for crop disease detection*. *Computers and Electronics in Agriculture*, 172, 105484.
5. PlantVillage Dataset. Available at: <https://plantvillage.psu.edu/>
6. Google Generative AI (Gemini API). Available at: <https://ai.google.dev/>
7. OpenWeatherMap API. Available at: <https://openweathermap.org/api>
8. Agmarknet API (Data.gov.in). Available at: <https://data.gov.in/>
9. MediaStack News API. Available at: <https://mediastack.com/>
10. Haifa Group Crop Nutrition Guide. Available at: <https://www.haifa-group.com/crop-guide>
11. Flask Web Framework. Available at: <https://flask.palletsprojects.com/>
12. PyTorch Deep Learning Framework. Available at: <https://pytorch.org/>
13. TailwindCSS Utility-First Framework. Available at: <https://tailwindcss.com/>
14. Government of India – Kisan Suvidha App. Available at: <https://farmer.gov.in/>
15. IFFCO Kisan App. Available at: <https://www.iffcokisan.com/>
16. Github Link - <https://github.com/DeepakYadav0047/Efarmer>